

УКРАЇНСЬКИЙ ДЕРЖАВНИЙ УНІВЕРСИТЕТ НАУКИ І ТЕХНОЛОГІЙ
ІННІ “ПРИДНІПРОВСЬКА ДЕРЖАВНА АКАДЕМІЯ БУДІВНИЦТВА ТА
АРХІТЕКТУРИ”

Кафедра комп’ютерних наук, інформаційних технологій та прикладної
математики

Лабораторна робота №1

З курсу “Нейронні мережі»

Варіант 13

Студентки третього курсу

Групи КН-23

спеціальності «Комп’ютерні науки»

Виконала: Селіхова Р. Р.

Перевірив: ст.в. Плахтій Євген Георгійович

м. Дніпро 2026

ЗМІСТ

ТЕОРЕТИЧНА ЧАСТИНА	3
ПРАКТИЧНА ЧАСТИНА	9
Перша частина:	9
ВИКОНАННЯ	10
1. Вибрати архітектуру та сформувати штучну нейронну мережу для вирішення задачі класифікації двоелементних образів на два класи.	10
2. Побудувати графік, який зображує простір входів, вхідні вектори та лінію, яка відображує, як навчена нейронна мережа розділяє простір входів на дві області.	12
3. Вибрати архітектуру та сформувати штучну нейронну мережу для вирішення задачі класифікації трьохелементних образів на два класи. .	13
4. Побудувати графік, який зображує простір входів, вхідні вектори та площину, яка відображує, як навчена нейронна мережа розділяє простір входів на дві області.	14
5. Вибрати архітектуру та сформувати штучну нейронну мережу для вирішення задачі класифікації двоелементних образів на чотири класи.	16
6. Побудувати графік, який зображує простір входів, вхідні вектори та лінії, яка відображує, як навчена нейронна мережа розділяє простір входів на чотири області.	17
Друга частина:	18
ВИКОНАННЯ	19
1. Вибрати архітектуру та сформувати штучну нейронну мережу для вирішення задачі класифікації:	19
2. Побудувати графік, який зображує простір входів та лінії, що відображають, як налаштована нейронна мережа ділить простір входів на підпростори, які відповідають різним класам.	21
3. Вибрати архітектуру та сформувати штучну нейронну мережу для вирішення задачі класифікації двоелементних вхідних векторів відповідно до рис. 2.1. Масштаб графіка вибрати самостійно.	22
КОНТРОЛЬНІ ЗАПИТАННЯ	26
ВИСНОВКИ	28
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	29

ТЕОРЕТИЧНА ЧАСТИНА

Для розв'язання будь-якої прикладної задачі за допомогою штучної нейронної мережі необхідно виконати п'ять етапів: підготувати дані для навчання, сформувати архітектуру ШНМ, ініціалізувати мережу, виконати навчання, протестувати навчену ШНМ.

Для вирішення задач класифікації вхідних векторів, які відносяться до класу лінійно роздільних, використовуються одношарові нейронні мережі, які називають персептрони.

Формування архітектури мережі.

Для створення ШНМ, яка має архітектуру одношарового персептрона, в пакеті Neural Network Toolbox використовується функція **perceptron**:

> net = perceptron (hardlimitTF, perceptronLF).

Вхідні аргументи:

hardlimitTF – активаційна функція (за замовчуванням '**hardlim**' – поро-гова);

perceptronLF – функція навчання (за замовчуванням '**learnp**').

Вихідні аргументи:

net – об'єкт класу Neural Network з архітектурою одношарового персептрона. Кількість нейронів персептрона визначається автоматично в процесі навчання, виходячи з того, що одношаровий персептрон, який містить m нейронів, здатен розділити вхідний простір на 2^m класів. Значення ваг та зміщень персептрона за замовчуванням ініціалізуються за допомогою функції **init**, яка встановлює нульові значення.

Існує інший спосіб сформувати одношаровий персептрон за допомогою функції **newp**:

> net = newp (X, T, s, hardlimitTF, perceptronLF).

Вхідні аргументи:

X – масив розмірністю $R \times Q$, який містить вхідні вектори, де R – кількість елементів, з яких складений вхідний вектор, Q – кількість векторів;

T – масив розмірністю $S \times Q$, який містить вихідні вектори, де S – кількість елементів, з яких складений вихідний вектор;

s – кількість нейронів.

Приклад. Сформувати персептрон для розподілу трьох 2-елементних векторів [0; 0], [1; 1] та [2; 2] на два класи: 0 та 1:

> X = [0 1 2; 0 1 2];

> T = [0 1 1];

> net = perceptron; % або

> net = newp (X, T, 'hardlim', 'learnp').

Ініціалізація мережі.

Після того як сформована архітектура мережі, необхідно задати початкові значення ваг та зміщень. Для цього можна використовувати:

а) функцію **init**, яка присвоює вагам та зміщенням нейронів

персептрона нульові значення:

> net = init (net);

б) функцію **rands**, яка генерує випадкові значення масивів ваг та зміщень:

> net.IW{1, 1} = rands (s, R) – генерує матрицю розмірністю $s \times R$, елементи якої приймають випадкові значення з інтервала $[-1; 1]$, та присвоює отримані значення матриці ваг нейронів першого шару персептрона;

> net.b{1} = rands (s) – генерує вектор розмірністю $s \times 1$, елементи якого приймають випадкові значення з інтервала $[-1; 1]$, та присвоює отримані значення вектору зміщень нейронів першого шару персептрона;

в) встановити бажані значення ваг та зміщень.

Приклад. Присвоїти вагам та зміщенню одношарового персептрона, який містить один нейрон, наступні значення: $w_1 = -3$, $w_2 = 2$, $w_0 = 5$.

> net.IW{1, 1} = [-3; 2];
> net.b{1} = 5.

Підготовка даних для навчання.

Вхідні вектори X можуть мати два формати: **cell array** та **double array**. Формат **cell array** відповідає *послідовному* способу подачі даних на вхід мережі, який можна інтерпретувати в такий спосіб. На вхід мережі подається послідовність з n вхідних сигналів, тобто на першому кроці – перший, на другому – наступний, і так далі. У відповідь мережа генерує послідовність виходів, яка також містить n елементів. Результат був би той самий, якби було створено n однакових мереж, і на кожну мережу було подано один із вхідних векторів.

Подання вхідних даних як числового масиву в форматі **double** відповідає *пакетному* способу представлення даних, в результаті чого на виході мережі генерується відповідний масив.

Приклад. Сформувати чотири види вхідних даних: один вектор-стовпець з двох елементів; група (пакет) з двох трьохелементних векторів; група з трьох двоелементних векторів; послідовність з трьох двоелементних векторів:

- 1) $X1 = [1; 2];$
- 2) $X2 = [1\ 4; 2\ 5; 3\ 6];$
- 3) $X3 = [1\ 3\ 5; 2\ 4\ 6];$
- 4) $X4 = \{[1; 2]\ [3; 4]\ [5; 6]\}.$

Навчання мережі.

Щоб підготувати нейронну мережу для вирішення конкретної задачі, необхідно налаштувати її параметри, тобто виконати навчання. У пакеті програм Neural Network Toolbox реалізовано два способи навчання: послідовний та пакетний (груповий).

Для організації послідовного навчання використовується функція **adapt**, яка коригує значення ваг та зміщень за результатами одноразової подачі на вхід мережі масиву вхідних даних:

> net_adapt = adapt (net, X, T).

Вхідні аргументи:

net – ім'я нейронної мережі до навчання;

X – масив вхідних векторів;

T – масив бажаних значень вихідних векторів (масив цілей).

Вихідні аргументи:

net_adapt – ім'я нейронної мережі після навчання.

Приклад. Реалізувати процедуру навчання нейронної мережі, використовуючи наступну послідовність навчаючих шаблонів

> $\{X_1 = \begin{bmatrix} -1 \\ -1 \end{bmatrix}, T_1 = 0\}, \{X_2 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, T_2 = 0\}, \{X_3 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, T_3 = 1\}, \{X_4 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, T_4 = 1\}$

> X = {[-1; -1] [-1; 1] [1; -1] [1; 1]};

> T = {0 0 1 1};

> net_1 = adapt (net, X, T).

Необхідно зауважити, що функція **adapt** виконує одноразове коригування параметрів мережі, тобто не гарантує, що після цього мережа виконає правильну класифікацію нового вхідного вектора. Для реалізації ітеративної процедури навчання необхідно організувати цикл, на кожному кроці якого застосовується процедура **adapt** та перевіряється, чи був виконаний критерій зупинки навчання (наприклад, поточний вектор похибок мережі містить нульові значення).

Для автоматичної реалізації ітеративної процедури навчання використовується функція **train**, синтаксис якої є аналогічним. При застосуванні функції **train** у відповідне вікно на моніторі виводиться схематичне зображення побудованої мережі та в реальному часі відображаються результати навчання: кількість виконаних епох навчання та значення помилки на виході мережі (за замовчуванням – середня абсолютна помилка). Якщо задача є нерозв'язною, тоді процес навчання буде зупинений тільки після виконання встановленої кількості епох (за замовчуванням 1000). Користувач може в будь-який момент зупинити процес навчання за допомогою кнопки «Stop Training», а також вивести на монітор графік залежності величини похибки від кількості епох навчання (кнопка «Performance»). Для зручності відображення результатів користувач може змінити логарифмічний масштаб по осі ординат (обраний за замовчуванням) на лінійний, вибравши в меню «Edit» пункт «Axes Properties».

Тестування (моделювання) мережі.

Після закінчення процесу навчання необхідно перевірити, наскільки успішно навчена ШНМ здатна вирішити конкретну задачу. Це можна зробити двома способами: за допомогою моделювання та шляхом графічного представлення результатів навчання.

Моделювання складається з подачі на вхід мережі тестових даних та порівняння результатів, які генерує мережа, з очікуваними. Для моделювання використовується функція **sim**:

```
> Y = sim (net_train, X).
```

Вхідні аргументи:

net_train – об'єкт класу Neural Network, навчання якого вже завершено;

X – масив, який містить вхідні вектори, призначені для тестування мережі.

Вихідні аргументи:

Y – масив, який містить вихідні вектори, що є результатом роботи ШНМ.

Для графічного відображення результатів навчання та моделювання використовуються функції **plotvec**, **plotpv**, **plotpc**.

Функція **plotvec** (X, color, Marker) відображає в двовимірному або в тривимірному просторі вектори-стовпці масиву X у вигляді маркерів, тип яких визначає строкова змінна Marker, а колір кожного маркера визначається елементами вектора-рядка color.

Приклад. Відобразити кружками різних кольорів чотири

двоелементних вхідних вектора:

```
> X = [1.0 1.3 1.7 2.0; 0 -1.0 0.5 -1.5];
```

```
> plotvec (X, [1 2 3 4], 'o').
```

Функція **plotpv** (X, T, v) відображає в двовимірному або в тривимірному просторі вектори-стовпці масиву входів X розміру R×Q у вигляді маркерів різного типу відповідно до значень масиву цілей T розміру S×Q; вектор-рядок v дозволяє задати границі відображуваного простору у форматі [x_min x_max y_min y_max].

Приклад. Відобразити маркерами різного типу чотири двоелементних вектора, які містить масив X, що належать двом класам:

```
> T = [0 0 1 1];
```

```
> plotpv (X, T, [0.5 2.5 -2 1]).
```

Результат представлено на рис. 1.1, а.

Функція **plotpc** (net.IW{1,1}, net.b{1}) будує лінію (в двовимірному просторі) або площину (в тривимірному), яка розділяє простір у відповідності до значень матриці ваг net.IW{1,1} та вектора зміщень net.b{1}

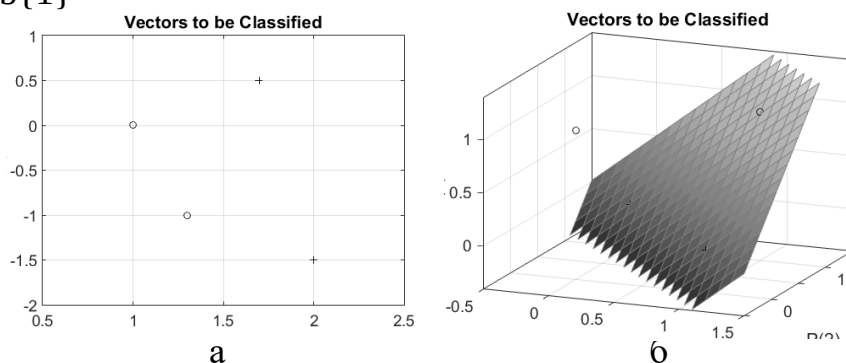


Рисунок 1.1

навченої нейронної мережі. Функцію **plotpc** доцільно використовувати разом з функцією **plotpv**, яка відображає розміщення в просторі вхідних векторів.

Приклад. Відобразити розміщення в тривимірному просторі

чотирьох вхідних векторів, що містяться в масиві X та належать двом класам, які визначаються масивом цілей T , а також площину, відповідно до якої одношаровий перцептрон, навчання якого проводилося за допомогою масивів X та T , розділяє простір на два класи:

```
> X = [0 0 1 1; 0 1 0 1; 1 0 0 1];
> T = [0 0 0 1];
> plotpv (X, T);
> hold on;
> plotpc (net.IW{1, 1}, net.b{1}).
```

Результат представлено на рис. 1.1, б.

Процедура навчання перцептрона має особливості. Одна з них полягає в наступному. Якщо значення елементів деякого вхідного вектора, що використовується при навчанні, значно відрізняються одне від одного, або від значень інших векторів, то для навчання може знадобитися чимало часу. Це обумовлено тим, що алгоритм навчання пов'язаний з додаванням (або відніманням) вхідного вектора до поточного вектору ваг. Таким чином, наявність вхідного вектора з дуже великими або малими елементами призводить до того, що настройка параметрів перцептрона вимагає дуже великої кількості ітерацій.

Врівноважити вплив великих чи малих компонент можна за допомогою масштабування вхідних даних. Необхідно нормувати дані так, щоб вплив будь-якого вхідного вектора мав приблизно рівний внесок. Отже, використання нормованого правила навчання перцептрона значно скорочує кількість циклів навчання, коли зустрічаються викиди вхідних векторів.

Для обчислення величини корекції матриці ваг та вектора зміщень перцептрона можуть застосовуватися дві функції: стандартна **'learnp'** та нормована **'learnpn'**. Нормована функція може бути вказана в якості аргументу при створенні перцептрона за допомогою функції **perceptron** або **newp** (за замовчуванням використовується стандартна функція).

Крім величини норми вхідних векторів, процедура навчання перцептрона є дуже чутливою до розміру навчаючої вибірки та порядку пред'явлення навчальних шаблонів. На практиці число навчальних шаблонів експоненціально зростає із зростанням кількості параметрів мережі, що настроюються. У книзі [4] наведено загальні рекомендації щодо розміру набору навчальних шаблонів. Рекомендується формувати навчальну вибірку так, щоб виконувалася наступна нерівність

Однак,

$$N > \frac{W}{\varepsilon}, \quad (2.1)$$

де N – кількість навчальних шаблонів, W – кількість параметрів мережі, що настроюються, ε – відсоток помилок, допустимий в ході тестування мережі.

В процесі навчання рекомендується переміжати вхідні образи, що належать до різних класів, а також «перетасовувати» (shuffling) навчальні шаблони. Один з можливих підходів полягає в тому, щоб на кожній ітерації алгоритму випадковим чином вибирати образ навчальної вибірки (тобто під час роботи алгоритму образи навчальної вибірки перебираються в випадковому порядку, можливо, неодноразово).

Якщо застосування нормованої функції '**learnpn**' не дозволяє успішно завершити процедуру навчання персептрона, рекомендується масштабувати вхідні дані таким чином, щоб значення векторів належали інтервалу $[0, 1]$ або $[-1, 1]$. Для цього необхідно знайти максимальне значення кожної з ознак розподілити значення о знаки кожного вектора на відповідні максимальні, де m – розмірність вектора входів: $x_i^{\text{norm}} = x_i / x_i^{\text{max}}$.

ПРАКТИЧНА ЧАСТИНА

Перша частина:

1. Вибрати архітектуру та сформувати штучну нейронну мережу для вирішення задачі класифікації двоелементних образів на два класи.

Набори навчальних шаблонів представлено в таблицях(13%4≈3):

Варіант 3

Образ	Клас
$[-1.5; 1.5]$	0
$[1.5; 2.0]$	0
$[-0.5; 0.3]$	1
$[2.0; 1.0]$	1

2. Побудувати графік, який зображує простір входів, вхідні вектори та лінію, яка відображує, як навчена нейронна мережа розділяє простір входів на дві області.
3. Вибрати архітектуру та сформувати штучну нейронну мережу для вирішення задачі класифікації трьохелементних образів на два класи.

Набори навчальних шаблонів представлено в таблицях:

Варіант 3

Образ	Клас
$[-1.0; -1.0; -1.0]$	0
$[1.0; -1.0; -1.0]$	1
$[-1.0; 1.0; -1.0]$	0
$[1.0; 1.0; -1.0]$	0
$[-1.0; -1.0; 1.0]$	1
$[1.0; -1.0; 1.0]$	1
$[-1.0; 1.0; 1.0]$	0
$[1.0; 1.0; 1.0]$	1

4. Побудувати графік, який зображує простір входів, вхідні вектори та площину, яка відображує, як навчена нейронна мережа розділяє простір входів на дві області.
5. Вибрати архітектуру та сформувати штучну нейронну мережу для вирішення задачі класифікації двохелементних образів на чотири класи.

Набори навчальних шаблонів представлено в таблицях:
Варіант 3

Образ	Клас
$[-0.5; 3.0]$	$[1; 0]$
$[0.4; 2.5]$	$[1; 0]$
$[-0.8; 0.3]$	$[0; 0]$
$[-1.0; -0.4]$	$[0; 0]$
$[-0.5; -0.1]$	$[0; 0]$
$[1.3; 0.5]$	$[1; 1]$
$[1.0; -0.2]$	$[1; 1]$
$[1.5; 0.8]$	$[1; 1]$
$[-0.5; -1.5]$	$[0; 1]$
$[0.2; -1.3]$	$[0; 1]$

6. Побудувати графік, який зображує простір входів, вхідні вектори та лінії, яка відображує, як навчена нейронна мережа розділяє простір входів на чотири області.

ВИКОНАННЯ

1. Вибрати архітектуру та сформувати штучну нейронну мережу для вирішення задачі класифікації двоелементних образів на два класи.

У першому завданні реалізовано класифікацію чотирьох двовимірних вхідних векторів на два класи за допомогою одношарового персептрона. Вхідна матриця X містить 4 вектори-стовпці розмірності 2×1 , а вектор цілей T задає належність кожного вектора до класу 0 або 1.

Персептрон формується за допомогою функції `newp`, яка приймає навчальну вибірку та масив цілей. Як активаційна функція використовується 'hardlim' (порогова), як функція навчання – 'learnp' (стандартне правило Розенблатта). Після ініціалізації мережі нульовими вагами за допомогою функції `init` задається ліміт в 1000 епох і викликається функція `train`, яка в ітеративному режимі коригує ваги та зміщення мережі.

Тестування виконується функцією `sim`, яка повертає прогнозовані класи для кожного вхідного вектора. Обчислюється кількість помилок як сума абсолютних відхилень між Y та T .

```

1      %% Підготовка навчальних даних
2      % Вхідні вектори (кожен стовпець – окремий образ)
3      X = [-1.5  1.5 -0.5  2.0;
4           1.5  2.0  0.3  1.0];
5
6      % Цільові значення класів для кожного вектора
7      T = [0  0  1  1];
8
9      %% Формування архітектури персептрона
10     % Формування одношарового персептрона за допомогою функції 'newp'
11     % 'hardlim' – активаційна функція, 'learnp' – функція навчання.
12     net = newp(X, T, 'hardlim', 'learnp');
13
14     %% Ініціалізація мережі
15     % Присвоєння вагам та зміщенням початкових нульових значень
16     net = init(net);
17
18     %% Навчання мережі
19     % Встановлення ліміту епох (за замовчуванням 1000)
20     net.trainParam.epochs = 1000;
21     % Функція train автоматично відкриває вікно з процесом навчання
22     net = train(net, X, T);
23
24     %% Тестування (моделювання) мережі
25     % Використовуємо функцію sim для отримання прогнозів мережі
26     Y = sim(net, X);
27
28     % Вивід результатів у командне вікно
29     fprintf('Очікувані класи (T): '); disp(T)
30     fprintf('Отримані класи (Y): '); disp(Y)
31
32     % Розрахунок кількості помилок
33     errors = sum(abs(Y - T));
34     fprintf('Кількість помилок після навчання: %d\n', errors);

```

Рисунок 1.1.1 – Код навчання мережі

Command Window

```

>> task1_part1
Очікувані класи (T):      0      0      1      1

Отримані класи (Y):      0      0      1      1

Кількість помилок після навчання: 0
>>

```

Рисунок 1.1.2 - Перевірка

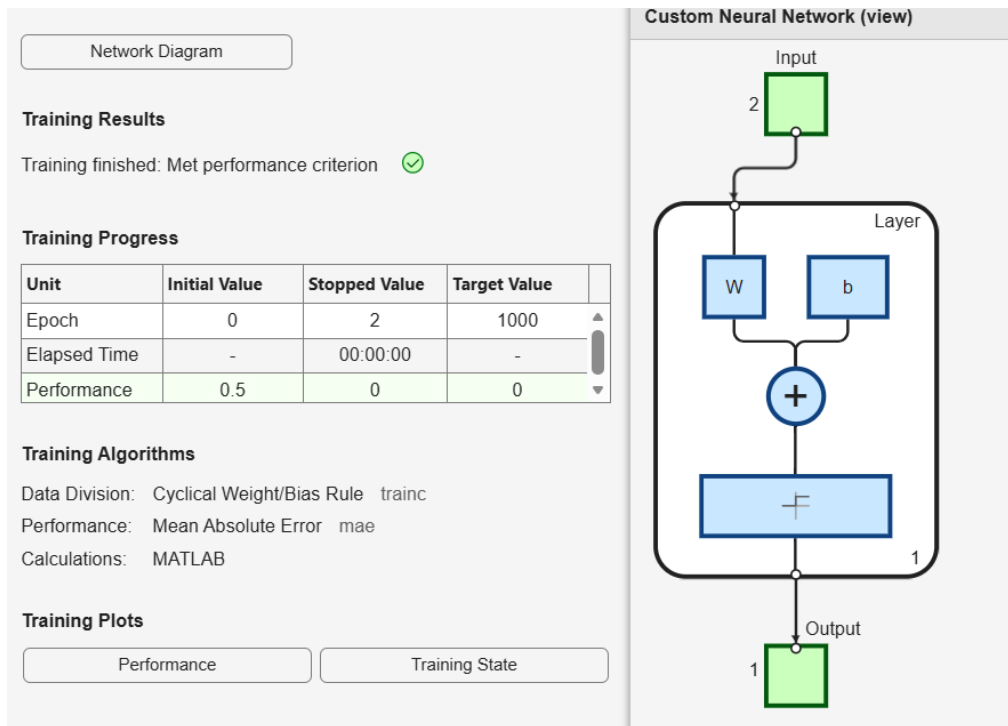


Рисунок 1.1.3 – Результати успішного навчання мережі та діаграма мережі

2. Побудувати графік, який зображує простір входів, входні вектори та лінію, яка відображує, як навчена нейронна мережа розділяє простір входів на дві області.

Для візуалізації використовується `plotpv` – відображає точки в площині з різними маркерами відповідно до їх класів. Функція `plotpc` будує розділяючу пряму на основі навчених ваг `net.IW{1,1}` та зміщення `net.b{1}`.

```

36 %% Побудова графіка (Візуалізація)
37 figure;
38 % Відображення входних образів на площині
39 plotpv(X, T);
40 hold on;
41 % Побудова лінії, що розділяє класи (на основі ваг та зміщення)
42 plotpc(net.IW{1,1}, net.b{1});
43
44 title('Класифікація 2D образів (одношаровий персептрон)');
45 xlabel('Ознака X1'); ylabel('Ознака X2');
46 legend('Клас 0', 'Клас 1', 'Розділяюча лінія');
47 grid on;

```

Рисунок 1.2.1 – Код графіку

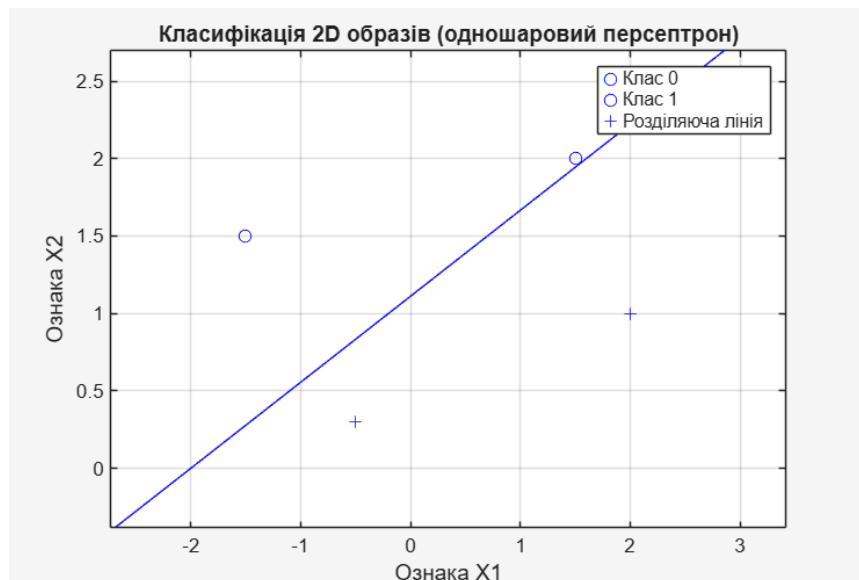


Рисунок 1.2.2 – Успішне розподілення входів на дві області

3. Вибрати архітектуру та сформувати штучну нейронну мережу для вирішення задачі класифікації трьохелементних образів на два класи.

У третьому завданні виконується класифікація восьми тривимірних образів вершин куба зі сторонами ± 1 на два класи. Вхідна матриця X має розмірність 3×8 , вектор цілей T містить бінарні мітки класів.

Персептрон формується функцією `newp` з явно заданими межами входів $PR = [-1 \ 1; -1 \ 1; -1 \ 1]$ та одним нейроном. Навчання виконується функцією `train` протягом 1000 епох. Після навчання порівнюються отримані та очікувані відповіді мережі.

```

1 % Підготовка навчальних даних
2 X = [-1.0  1.0  -1.0  1.0  -1.0  1.0  -1.0  1.0;
3      -1.0 -1.0   1.0  1.0  -1.0 -1.0   1.0  1.0;
4      -1.0 -1.0  -1.0 -1.0   1.0  1.0   1.0  1.0];
5
6 T = [0  1  0  0  1  1  0  1];
7
8 % Формування персептрона
9 % newp(PR, S) де PR – межі входів [min max] для кожної ознаки
10 PR = [-1 1; -1 1; -1 1]; % межі для 3 ознак
11 net = newp(PR, 1);      % 1 нейрон
12
13 % Ініціалізація
14 net = init(net);
15 net.trainParam.epochs = 1000;
16
17 % Навчання
18 net = train(net, X, T);
19
20 % Тестування
21 Y = net(X);
22 fprintf('Очікувані: '); disp(T)
23 fprintf('Отримані: '); disp(Y)
24 fprintf('Помилка: %d\n', sum(Y ~= T));

```

Рисунок 1.3.1 – Створення мережі для трьохелементних образів

```
>> task1_part3
Очікувані:    0    1    0    0    1    1    0    1

Отримані:     0    1    0    0    1    1    0    1

Помилки: 0
```

Рисунок 1.3.2 - Перевірка

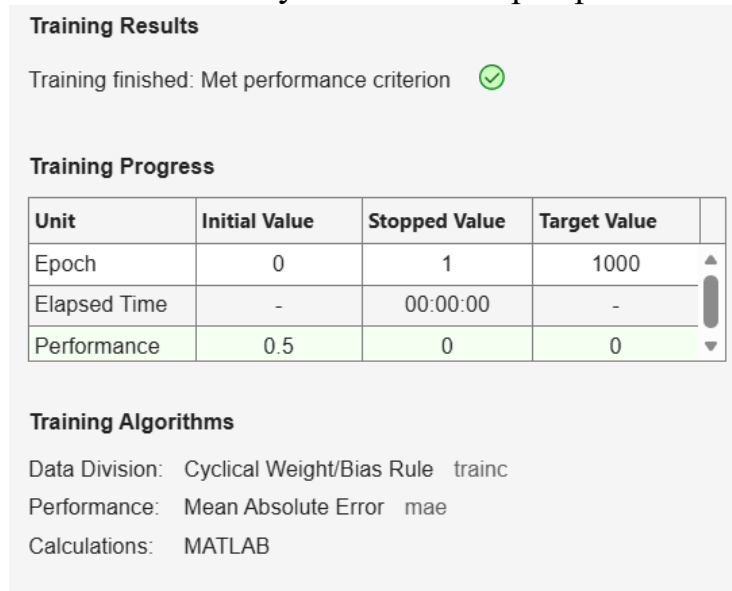


Рисунок 1.3.3 – Результати успішного навчання мережі

4. Побудувати графік, який зображує простір входів, вхідні вектори та площину, яка відображує, як навчена нейронна мережа розділяє простір входів на дві області.

Оскільки простір вхідних даних тривимірний, функція `plotpv` відображає хмару точок у 3D-просторі. Функція `plotpc` будує розділяючу площину, що ділить тривимірний простір на два класи відповідно до навчених параметрів мережі.

```
26 %% Візуалізація
27 figure;
28 plotpv(X, T);
29 hold on;
30 plotpc(net.IW{1,1}, net.b{1});
31 title('Класифікація 3D образів');
32 xlabel('X1'); ylabel('X2'); zlabel('X3');
33 grid on;
```

Рисунок 1.4.1 – Код графіку

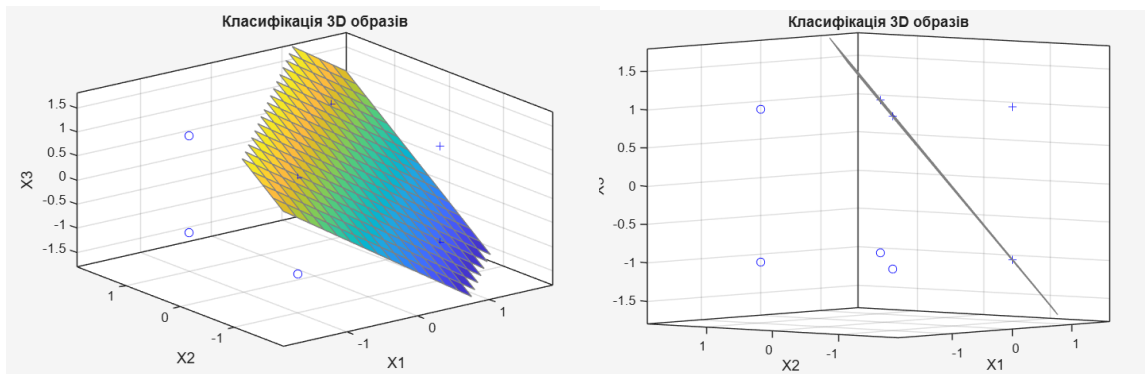


Рисунок 1.4.2 - Успішне розподілення входів на дві області площиною

Проте графік показує, що три точки наближені до площини або лежать на ній. Додамо перевірку.

```

26 %% Перевірка точок
27 % Отримуємо ваги та зсув з навченої мережі
28 W = net.IW{1,1}; % Вектор ваг (1x3)
29 b = net.b{1}; % Зсув (скаляр)
30
31 % Норма вектора ваг
32 norm_W = norm(W);
33
34 % Допуск для порівняння з нулем
35 tolerance = 1e-5;
36
37 % Масив точок для перевірки
38 points_to_check = X;
39
40 fprintf('\n--- Аналіз розташування точок ---\n');
41 for i = 1:size(points_to_check, 2)
42     pt = points_to_check(:, i);
43
44     % Формула відстані від точки до площини
45     distance = abs(W * pt + b) / norm_W;
46
47     % Перевірка та текстовий висновок
48     if distance <= tolerance
49         fprintf('Точка %d (%.1f, %.1f, %.1f) лежить НА розділювальній площині.\n', ...
50             i, pt(1), pt(2), pt(3));
51     else
52         fprintf('Точка %d (%.1f, %.1f, %.1f) не лежить на площині. Відстань до неї: %.4f\n', ...
53             i, pt(1), pt(2), pt(3), distance);
54     end
55 end

```

Рисунок 1.4.3 – Код перевірки

```

Command Window
--- Аналіз розташування точок ---
Точка 1 (-1.0, -1.0, -1.0) не лежить на площині. Відстань до неї: 1.1547
Точка 2 (1.0, -1.0, -1.0) лежить НА розділювальній площині.
Точка 3 (-1.0, 1.0, -1.0) не лежить на площині. Відстань до неї: 2.3094
Точка 4 (1.0, 1.0, -1.0) не лежить на площині. Відстань до неї: 1.1547
Точка 5 (-1.0, -1.0, 1.0) лежить НА розділювальній площині.
Точка 6 (1.0, -1.0, 1.0) не лежить на площині. Відстань до неї: 1.1547
Точка 7 (-1.0, 1.0, 1.0) не лежить на площині. Відстань до неї: 1.1547
Точка 8 (1.0, 1.0, 1.0) лежить НА розділювальній площині.
>>

```

Рисунок 1.4.4 – Результат

Бачимо що три входи лежать прямо на площині.

Функція `hardlim` визначає клас так:

$y = 1$, якщо $w \cdot x + b \geq 0$

$y = 0$, якщо $w \cdot x + b < 0$

Коли точка лежить на площині, то $w \cdot x + b = 0$, і вона отримує клас 1 (за рахунок $\geq$). Якщо для цієї точки ціль теж 1 – помилки немає, навчання завершується успішно.

5. Вибрати архітектуру та сформувати штучну нейронну мережу для вирішення задачі класифікації двохелементних образів на чотири класи.

Оскільки один нейрон з функцією `hardlim` поділяє простір лише на 2 класи, для розподілу на 4 класи використовується перцептрон з двома нейронами ($2^2 = 4$ можливих класи).

Вихідна матриця цілей T має розмірність 2×10 – кожен стовпець є бінарним вектором $[0;0]$, $[0;1]$, $[1;0]$ або $[1;1]$, що кодує один із чотирьох класів. Перцептрон формується функцією `perceptron` без явного задання кількості нейронів – вона визначається автоматично на основі розмірності матриці T .

```

1 % Навчальні дані
2 X = [-0.5  0.4 -0.8 -1.0 -0.5  1.3  1.0  1.5 -0.5  0.2;
3       3.0  2.5  0.3 -0.4 -0.1  0.5 -0.2  0.8 -1.5 -1.3];
4
5 T = [1  1  0  0  0  1  1  1  0  0;
6       0  0  0  0  0  1  1  1  1  1];
7
8 % Формування перцептрона
9 net = perceptron;
10
11 % Навчання за допомогою train
12 net.trainParam.epochs = 1000;
13 net = train(net, X, T);
14
15 % Тестування за допомогою sim
16 Y = sim(net, X);
17 disp('Очікувані:'); disp(T)
18 disp('Отримані:');  disp(double(Y))
19 fprintf('Помилка: %d\n', sum(any(Y ~= T)));
20

```

Рисунок 1.5.1 – Код мережі двохелементних образів на чотири класи

```

>> task1_part5
Очікувані:
    1    1    0    0    0    1    1    1    0    0
    0    0    0    0    0    1    1    1    1    1

Отримані:
    1    1    0    0    0    1    1    1    0    0
    0    0    0    0    0    1    1    1    1    1

Помилка: 0

```

Рисунок 1.5.2 - Перевірка

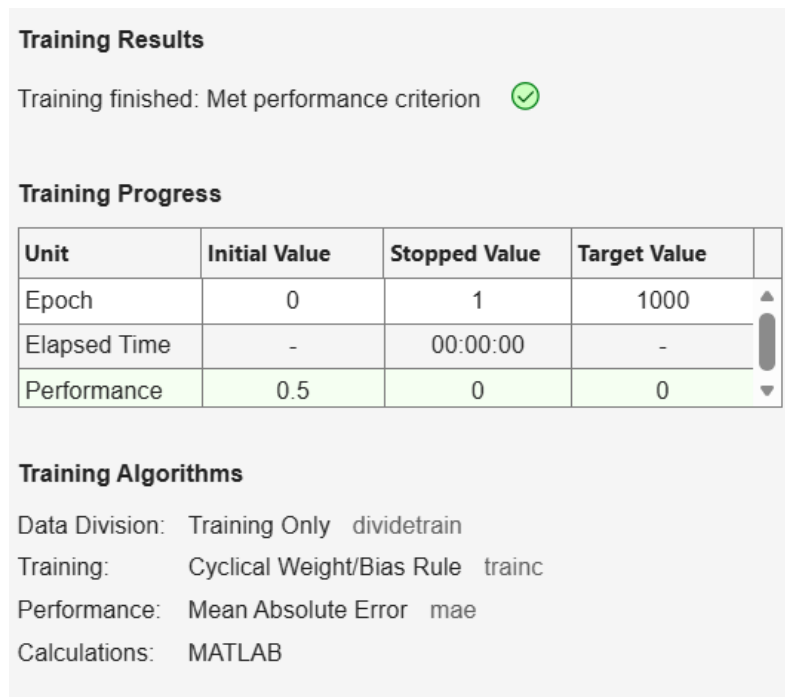


Рисунок 1.5.3 – Результат успішно навченої мережі

6. Побудувати графік, який зображує простір входів, вхідні вектори та лінії, яка відображує, як навчена нейронна мережа розділяє простір входів на чотири області.

Після навчання на графіку відображаються дві розділяючі прямі по одній для кожного нейрона які разом ділять площину на чотири підобласті. Кожна пряма будується функцією `plotpc` з відповідним рядком матриці ваг і відповідним елементом вектора зміщень.

```

21 % Візуалізація
22 figure;
23 plotpv(X, T);
24 hold on;
25 plotpc(net.IW{1,1}(1,:), net.b{1}(1));
26 plotpc(net.IW{1,1}(2,:), net.b{1}(2));
27 title('Класифікація на 4 класи');
28 xlabel('X(1)'); ylabel('X(2)');
29 grid on;

```

Рисунок 1.6.1 – Код графіку

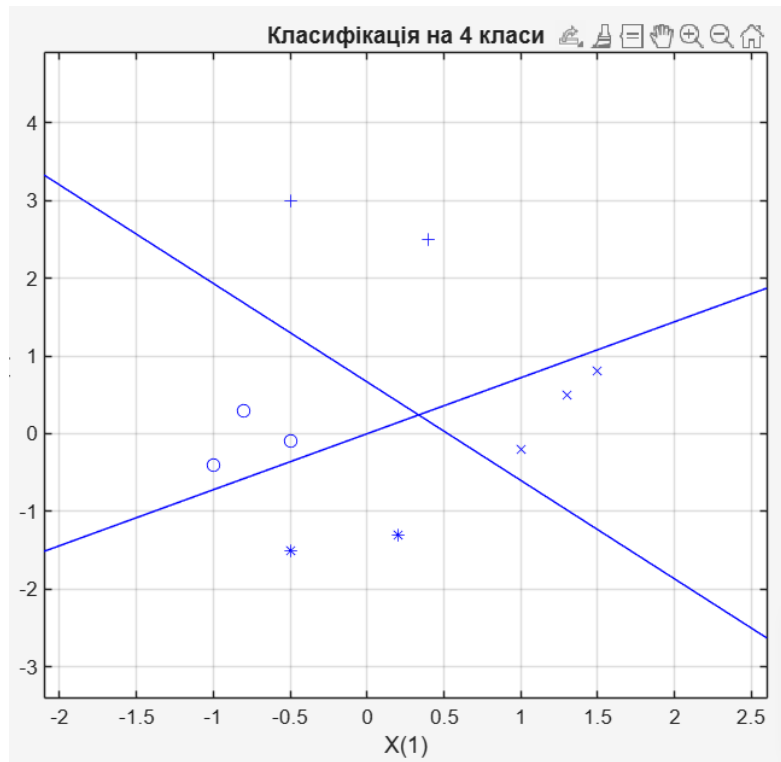


Рисунок 1.6.2 – Успішне розподілення входів на чотири області

Друга частина:

1. Вибрати архітектуру та сформувати штучну нейронну мережу для вирішення задачі класифікації:

Варіант 3: літературні твори за двома ознаками (кількість головних героїв та кількість сторінок) на два класи:

- 1) повісті – кількість головних героїв не більше 5 та кількість сторінок не перевищує 200;
- 2) романи – кількість головних героїв більше 5 та кількість сторінок понад 200.

Для вирішення задачі необхідно:

- сформувати навчальну вибірку, яка містить достатню кількість шаблонів, кожен з яких включає вхідний вектор та відповідний вихідний вектор; сформувати персептрон відповідної розмірності, вказавши в якості функції навчання **'learnnpn'**;
 - реалізувати процес навчання персептрона за допомогою функції **adapt** або **train**;
 - шляхом моделювання перевірити результат роботи навченої мережі.
2. Побудувати графік, який зображує простір входів та лінії, що відображають, як налаштована нейронна мережа ділить простір входів на підпростори, які відповідають різним класам.
 3. Вибрати архітектуру та сформувати штучну нейронну мережу

для вирішення задачі класифікації двоелементних вхідних векторів відповідно до рис. 2.1. Масштаб графіка вибрати самостійно.

$x(2)$

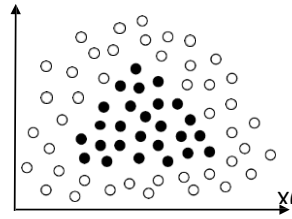


Рисунок 2.1

4. Перевірити працездатність навченої мережі за допомогою моделювання.

ВИКОНАННЯ

1. Вибрати архітектуру та сформувати штучну нейронну мережу для вирішення задачі класифікації:

Варіант 3: літературні твори за двома ознаками (кількість головних героїв та кількість сторінок) на два класи:

- 0) повісті – кількість головних героїв не більше 5 та кількість сторінок не перевищує 200;
- 1) романи – кількість головних героїв більше 5 та кількість сторінок понад 200.

Оскільки ознаки мають суттєво різні масштаби (кількість героїв: 1–12, кількість сторінок: 30–600), виконується нормування: кожна ознака ділиться на своє максимальне значення у вибірці за формулою $x_{norm} = x / x_{max}$, що приводить усі значення до інтервалу $[0, 1]$. Це є обов'язковою умовою для коректної роботи нормованої функції навчання `learnnp`.

Персептрон формується за допомогою функції `perceptron` з параметрами: активаційна функція – `'hardlim'` (жорстке порогове обмеження), функція навчання – `'learnnp'` (нормоване правило Розенблатта). Кількість нейронів визначається автоматично відповідно до розмірності вектора цілей T – у даному випадку один нейрон (бінарна класифікація).

Ліміт навчання встановлено в 1000 епох (`net.trainParam.epochs = 1000`). Навчання виконується функцією `train`, яка в ітеративному режимі коригує ваги та зміщення мережі до досягнення нульової помилки або вичерпання ліміту епох.

Після навчання виконується тестування за допомогою функції `sim`, яка повертає вихідний вектор Y із прогнозованими класами. Кількість помилок обчислюється як $\text{sum}(Y \sim T)$. За умови успішного навчання на нормованих даних мережа демонструє нульову кількість помилок на навчальній вибірці – усі 20 зразків класифіковані вірно.

Нормування вхідних даних є ключовим чинником у даному завданні: без нього нормована функція `learnprn` не може стабілізувати навчання через відмінність масштабів ознак, що призводить до збільшення числа ітерацій або взагалі до невдалого навчання.

```

1 % Навчальна вибірка
2 % Повісті (клас 0): до 5 героїв, до 200 сторінок
3 X_novel = [2 3 4 5 1 3 5 4 2 5;
4            50 80 120 180 30 150 200 90 60 170];
5
6 % Романи (клас 1): більше 5 героїв, більше 200 сторінок
7 X_roman = [6 7 8 10 9 6 12 8 7 11;
8            250 300 400 500 350 220 600 280 320 450];
9
10 X = [X_novel, X_roman];
11 T = [zeros(1,10), ones(1,10)];
12
13 % Нормування
14 X_norm = [X(1,:)/max(X(1,:)); X(2,:)/max(X(2,:))];
15
16 % Формування персептрона з нормованою функцією навчання
17 net = perceptron('hardlim', 'learnprn');
18 net.trainParam.epochs = 1000;
19
20 % Навчання за допомогою train
21 net = train(net, X_norm, T);
22
23 % Тестування за допомогою sim
24 Y = sim(net, X_norm);
25 fprintf('Помилка: %d / %d\n', sum(Y ~= T), length(T));
26

```

Рисунок 2.1.1 – Код мережі для вирішення задачі

Command Window

```
>> task2_part1
Помилка: 0 / 20
```

Рисунок 2.1.2 - Перевірка

Training Results

Training finished: Met performance criterion ✓

Training Progress

Unit	Initial Value	Stopped Value	Target Value	
Epoch	0	5	1000	▲
Elapsed Time	-	00:00:00	-	▬
Performance	0.5	0	0	▼

Training Algorithms

Data Division: Training Only `dividettrain`
 Training: Cyclical Weight/Bias Rule `trainc`
 Performance: Mean Absolute Error `mae`
 Calculations: MATLAB

Рисунок 2.1.3 – Результат успішного навчання мережі

2. Побудувати графік, який зображує простір входів та лінії, що відображають, як налаштована нейронна мережа ділить простір входів на підпростори, які відповідають різним класам.

Для візуалізації результатів навчання використовуються стандартні функції пакету Neural Network Toolbox: `plotpv` та `plotpc`.

Функція `plotpv(X_norm, T)` відображає нормовані вхідні точки у двовимірному просторі ознак: нормована кількість героїв (вісь X) та нормована кількість сторінок (вісь Y). Точки двох класів позначаються різними маркерами – кружки для класу 0 (повісті) та хрестики для класу 1 (романи).

Функція `plotpc(net.IW{1,1}, net.b{1})` будує розділяючу гіперплощину (пряму лінію у двовимірному просторі) на основі навчених параметрів мережі: матриці ваг `net.IW{1,1}` та вектора зміщень `net.b{1}`. Ця лінія є геометричним відображенням рівняння персептрона: $w_1 \cdot x_1 + w_2 \cdot x_2 + b = 0$.

На отриманому графіку чітко видно, що всі точки повістей (клас 0) зосереджені в лівому нижньому куті простору ознак – мають малу нормовану кількість героїв та невелику нормовану кількість сторінок. Натомість романи (клас 1) розташовані у правому верхньому куті. Розділяюча пряма успішно відокремлює два класи, жодна точка навчальної вибірки не знаходиться по «неправильний» бік лінії, що підтверджує нульову кількість помилок після навчання.

Завдання наочно демонструє лінійну роздільність класів: повісті та романи чітко кластеризуються у різних зонах двовимірного простору ознак, тому одношаровий персептрон з одним нейроном виявляється достатнім для їх розділення. Нормування при цьому не змінює геометричного розташування точок відносно одна одної, але забезпечує числову стабільність алгоритму навчання.

```
27 %% Візуалізація
28 figure;
29 plotpv(X_norm, T);
30 hold on;
31 plotpc(net.IW{1,1}, net.b{1});
32 title('Повісті vs Романи');
33 xlabel('Герої (норм.)'); ylabel('Сторінки (норм.)');
34 grid on;
```

Рисунок 2.2.1 – Код графіку

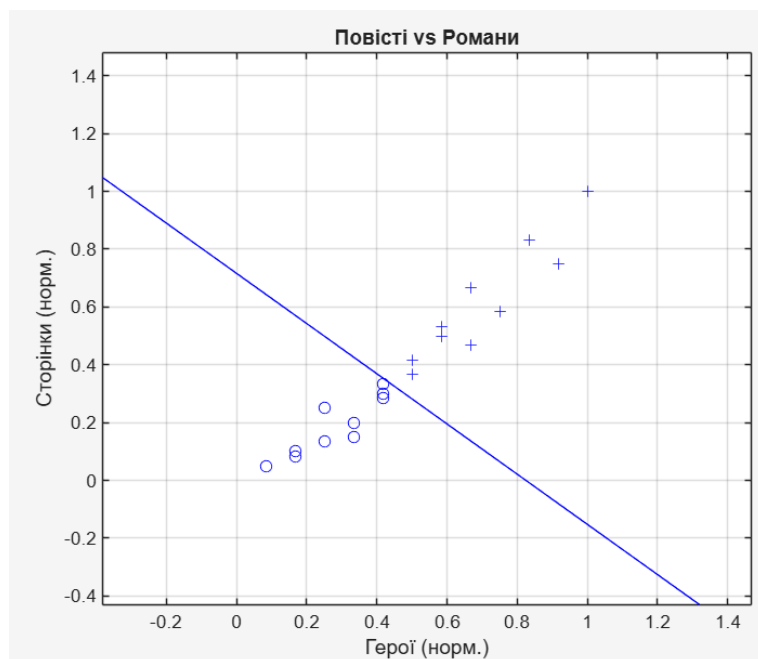


Рисунок 2.2.2 – Успішне розподілення входів на дві області

3. *Вибрати архітектуру та сформувати штучну нейронну мережу для вирішення задачі класифікації двоелементних входних векторів відповідно до рис. 2.1. Масштаб графіка вибрати самостійно.*

У третьому завданні другої частини реалізовано класифікацію просторово складного розподілу точок: центральна група «чорних» точок оточена трьома групами «білих» точок, розташованих у різних кутах площини. Така конфігурація є лінійно нероздільною – жодна одна пряма не може відокремити чорні точки від усіх білих одночасно.

Навчальна вибірка містить 10 чорних точок, зосереджених поблизу центру координат у діапазоні приблизно $[-0.5, 0.5] \times [0.0, 0.6]$, та 16 білих точок, розподілених у трьох групах: BL (нижній лівий кут), BR (нижній правий кут) та T (верхня зона).

Задача вирішується за допомогою перцептрона з трьома нейронами. Кожен нейрон відповідає за відокремлення центральної групи від однієї конкретної групи білих точок: нейрон 1 — від групи BL (нижній лівий кут), нейрон 2 – від групи BR (нижній правий кут), нейрон 3 – від групи T (верхня зона). Перцептрон формується за допомогою функції `newp(X, T, 'hardlim', 'learnprn')`, де X – матриця 2×26 з координатами всіх точок.

Матриця цілей T має розмірність 3×26 – три рядки відповідають трьом нейронам, а 26 стовпців – усім точкам вибірки. Стратегія кодування цілей побудована наступним чином: кожен нейрон видає значення 1 для всіх точок, крім "своїї" групи білих точок, для якої він видає 0. Чорні точки завжди отримують вихід $[1; 1; 1]$ від усіх трьох нейронів – саме ця комбінація є

ознакою приналежності до центрального класу. Кожна з груп білих точок має рівно один нульовий вихід у відповідному нейроні.

Навчання виконується функцією `train` з лімітом 1000 епох. Після навчання помилка перевіряється по всіх стовпцях: помилка фіксується тоді, коли хоча б один із трьох нейронів видає неправильне значення для певної точки – `sum(any(Y ~= T))`. Успішне навчання дає нуль помилок.

```

1 % Підготовка даних
2 % Чорні точки (Центральна група)
3 X_black = [ 0.0 0.3 -0.3 0.5 -0.5 0.2 -0.2 0.4 -0.4 0.0;
4             0.5 0.4 0.4 0.2 0.2 0.6 0.6 0.0 0.0 0.3];
5 num_black = size(X_black, 2);
6
7 % Білі точки (Три групи "охоронців")
8 X0_BL = [-1.5 -1.2 -1.0 -1.3 -0.9; -1.0 -0.8 -1.2 -0.3 -0.5]; % Низ-ліво
9 X0_BR = [ 1.2 1.5 1.0 1.3 0.9; -1.0 -0.8 -1.1 -0.3 -0.5]; % Низ-право
10 X0_T = [ 0.0 0.3 -0.3 0.5 -0.5 0.2; 1.5 1.8 1.7 1.6 1.4 1.3]; % Верх
11
12 X_white = [X0_BL, X0_BR, X0_T];
13 num_white = size(X_white, 2);
14
15 % Об'єднуємо координати
16 X = [X_black, X_white];
17
18 % Формування цілей (T) для 3-х нейронів
19 % Кожен нейрон має видати "0" тільки для "своїх" групи білих точок
20 T_black = ones(3, num_black);
21 T_white = [zeros(1,5), ones(1,5), ones(1,6); % Нейрон 1 "відсікає" BL
22            ones(1,5), zeros(1,5), ones(1,6); % Нейрон 2 "відсікає" BR
23            ones(1,5), ones(1,5), zeros(1,6)]; % Нейрон 3 "відсікає" T
24 T = [T_black, T_white];
25
26 % Створення та навчання мережі
27 net = newp(X, T, 'hardlim', 'learnpn');
28 net.trainParam.epochs = 1000;
29 net = train(net, X, T);
30
31 % Тестування
32 Y = sim(net, X);
33 errors = sum(any(Y ~= T));
34 fprintf('Помилки: %d / %d\n', errors, size(T,2));
35

```

Рисунок 2.3.1 – Створення та навчання мережі

```

36 %% Візуалізація
37 figure('Color', 'w'); % Білий фон вікна
38
39 plot(X_black(1,:), X_black(2,:), 'ko', 'MarkerFaceColor', 'k', 'MarkerSize', 8);
40 hold on;
41
42 plot(X_white(1,:), X_white(2,:), 'ko', 'MarkerFaceColor', 'w', 'MarkerSize', 8, 'LineWidth', 1.5);
43
44 % Малюємо лінії нейронів
45 for i = 1:3
46     plotpc(net.IW{1,i}(1,:), net.b{1,i});
47     h = findobj(gca, 'Type', 'line');
48     set(h(1), 'Color', 'k', 'LineWidth', 1);
49 end
50
51 title('Класифікація (три нейрони)');
52 xlabel('x1'); ylabel('x2');
53 grid on;
54
55 % Масштаб
56 axis tight;
57 pad = 0.5;
58 axis([min(X(1,:))-pad max(X(1,:))+pad min(X(2,:))-pad max(X(2,:))+pad]);

```

Рисунок 2.3.2 – Код графіку

Command Window

```
>> task2_part3
Помилки: 0 / 26
```

Рисунок 2.3.3 – Перевірка

На графіку візуалізовано результат: чорні точки у центрі позначені заповненими кружками, білі точки – порожніми кружками. Три розділяючі прямі будуються функцією `plotps` окремо для кожного нейрона і відображаються чорними лініями. Разом вони утворюють трикутну зону навколо центральної групи точок, ефективно ізолюючи «чорні» точки від усіх трьох груп «білих».

Геометрично кожна пряма відсікає одну з груп білих точок, утворюючи разом тришарову захисну межу для центрального класу. Точка вважається «чорною» (класифікованою як центральна), якщо і тільки якщо всі три нейрони дають значення 1 – тобто точка знаходиться «всередині» всіх трьох розділяючих прямих.

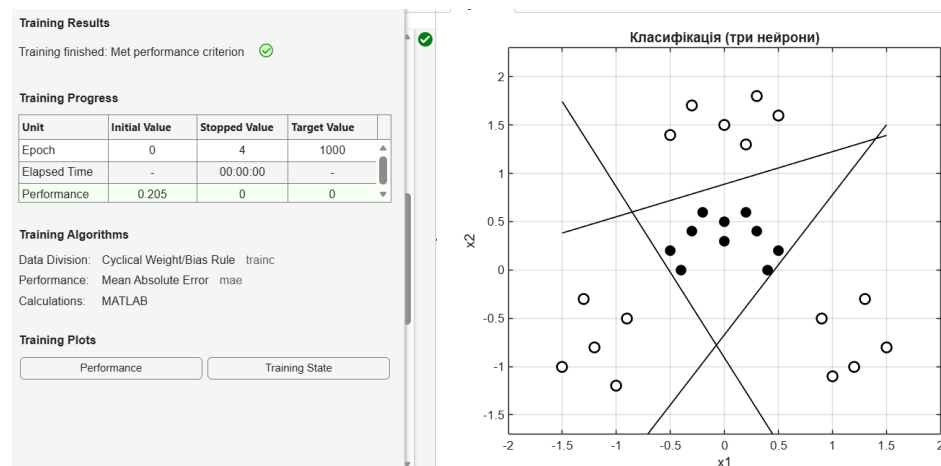


Рисунок 2.3.4 – Успішне навчання та розподілення входів обох типів на окремі області

4. Перевірити працездатність навченої мережі за допомогою моделювання.

Для перевірки працездатності навченої мережі виконується моделювання на трьох нових тестових точках, яких не було у навчальній вибірці. Тестові точки задаються матрицею X_{test} розмірністю 2×3 :

Точка 1: $[0.1; 0.3]$ – знаходиться поблизу центру, серед чорних точок.

Точка 2: $[-1.2; -0.5]$ – знаходиться в нижньому лівому куті, поблизу групи ВЛ білих точок.

Точка 3: $[1.4; 1.5]$ – знаходиться у верхньому правому куті, поблизу групи Т білих точок.

Моделювання виконується функцією `sim(net, X_test)`, яка повертає матрицю виходів Y_{test} розмірністю 3×3 – три виходи трьох нейронів для кожної з трьох тестових точок. Класифікація визначається за правилом: якщо для певної точки `all(Y_test(:, j) == 1)`, тобто всі три нейрони видають 1, точка класифікується як «чорна» (центральна). В іншому разі вона відноситься до «білих».

Очікувані результати моделювання: точка $[0.1; 0.3]$ класифікується як ЧОРНА – вона розташована всередині трикутної зони; точки $[-1.2; -0.5]$ та $[1.4; 1.5]$

класифікуються як БІЛІ – вони знаходяться поза захисною зоною, в областях груп ВЛ та Т відповідно.

```

59 % Моделювання на нових даних
60 disp('--- Тестування мережі (Моделювання) ---');
61
62 % Створимо кілька нових тестових точок, яких не було в навчанні
63 % X_test(1,:) - координата x1
64 % X_test(2,:) - координата x2
65 X_test = [0.1, -1.2, 1.4;
66           0.3, -0.5, 1.5];
67
68 % Проводимо моделювання (отримуємо виходи 3-х нейронів для кожної точки)
69 Y_test = sim(net, X_test);
70
71 % Візуалізуємо нові точки на існуючому графіку
72 figure(1); hold on;
73
74 for j = 1:size(X_test, 2)
75     % Точка вважається "Чорною" (належить до центральної групи) ТІЛЬКИ ЯКЩО
76     % всі 3 нейрони видали '1' (тобто Y_test(:, j) == [1; 1; 1]).
77     if all(Y_test(:, j) == 1)
78         fprintf('Тестова точка [%1f; %1f] класифікована як: ЧОРНА\n', X_test(1,j),
79               % Малюємо тестову точку зеленим квадратом, якщо вона класифікована як чорна
80               plot(X_test(1,j), X_test(2,j), 'gs', 'MarkerSize', 10, 'LineWidth', 2);
81     else
82         fprintf('Тестова точка [%1f; %1f] класифікована як: БІЛА\n', X_test(1,j),
83               % Малюємо тестову точку червоним хрестиком, якщо вона класифікована як біла
84               plot(X_test(1,j), X_test(2,j), 'rx', 'MarkerSize', 10, 'LineWidth', 2);
85     end
86 end
87
88 title('Класифікація та Моделювання');

```

Рисунок 2.4.1 – Код моделювання нових даних

Тестові точки відображаються на існуючому графіку: зелений квадрат (маркер 'gs') позначає точки, класифіковані як «чорні», а червоний хрестик (маркер 'rx') – точки, класифіковані як «білі». Це дозволяє наочно перевірити відповідність результатів моделювання геометричному розташуванню точок на площині відносно розділяючих ліній.

Результати моделювання підтверджують коректну роботу навченої мережі: точки з нових даних класифікуються відповідно до просторового положення – всередині чи поза трикутною зоною захисту центральної групи. Це демонструє здатність мережі до узагальнення – правильної класифікації нових, раніше не бачених прикладів.

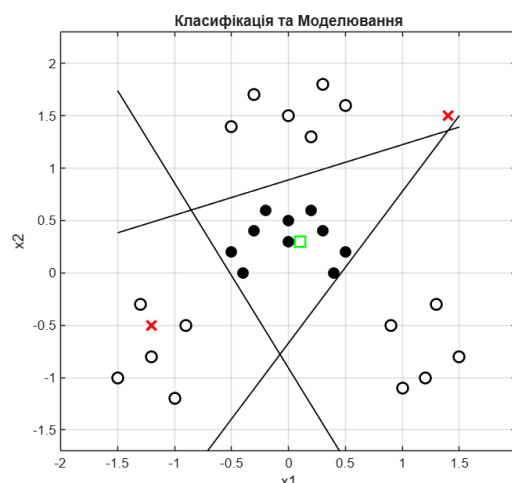


Рисунок 2.4.2 – Графік результату моделювання

Посилання на GitHub: https://github.com/baldezhkkk/NN_Assignment1

КОНТРОЛЬНІ ЗАПИТАННЯ

1. В яких випадках для навчання персептрона доцільно застосовувати нормовану функцію навчання?

Використання нормованого правила навчання (функція `learnnpn`) доцільне, коли:

- Значення елементів вхідних векторів значно відрізняються одне від одного або від значень інших векторів.
- У вхідних даних зустрічаються «викиди» (різко відмінні значення).
- Необхідно скоротити кількість циклів навчання та зрівноважити вплив різних компонентів, щоб кожен вхідний вектор мав приблизно рівний внесок.

2. Чи впливає на результат навчання персептрона послідовність пред'явлення навчальних шаблонів?

Так, впливає, бо процедура навчання персептрона є дуже чутливою до порядку пред'явлення навчальних шаблонів. Саме тому в процесі навчання рекомендується «перетасовувати» (`shuffling`) шаблони або вибирати їх випадковим чином на кожній ітерації.

3. Чи можна добитися розділення вхідних образів відповідно до рис. 2.1 за допомогою персептрона, що містить нейрони з нульовими значеннями зміщень?

Згідно з текстом та рис. 2.1, образи розділені складною межею (чорні точки оточені білими), що вказує на нелінійну роздільність класів. Одношаровий персептрон призначений для вирішення задач класифікації лише лінійно роздільних векторів. Навіть за умови лінійної роздільності, нульове зміщення ($b=0$) змушує розділяючу лінію проходити через початок координат, що критично обмежує можливості налаштування мережі. Для розділення образів, як на рис. 2.1, одношарового персептрона (навіть зі зміщенням) буде недостатньо.

4. Яка активаційна функція використовується для нейронів, що утворюють персептрон?

Для нейронів персептрона за замовчуванням використовується активаційна функція 'hardlim' (функція жорсткого порогу).

5. Яка кількість нейронів, об'єднаних в одношаровий персептрон, знадобиться, щоб розділити вхідні образи на 10 класів?

Кількість нейронів m визначає кількість класів як 2^m .

- Якщо $m=3$, можна розділити на $2^3 = 8$ класів (недостатньо).
- Якщо $m=4$, можна розділити на $2^4 = 16$ класів.

Отже, для розділення на 10 класів знадобиться 4 нейрони.

6. Чи може одношаровий персептрон реалізувати логічну операцію XOR («виключає АБО»)? Обґрунтуйте свою відповідь.

Ні, не може. Одношарові персептрони використовуються для класифікації тільки тих вхідних векторів, які відносяться до класу лінійно роздільних. Операція XOR є класичним прикладом лінійно нероздільної задачі (неможливо провести одну пряму лінію, яка б відокремила точки $[0,0]$ і $[1,1]$ від точок $[0,1]$ і $[1,0]$).

ВИСНОВКИ

У ході виконання лабораторної роботи було вивчено та практично реалізовано принципи роботи одношарового персептрона для вирішення задач класифікації образів засобами пакету Neural Network Toolbox у середовищі MATLAB.

У першій частині роботи послідовно розв'язано три задачі: класифікація двовимірних образів на два класи (завдання 1), класифікація тривимірних образів на два класи з побудовою розділяючої площини (завдання 3), та класифікація двовимірних образів на чотири класи за допомогою дворівневого персептрона (завдання 5). Результати підтвердили, що одношаровий персептрон з m нейронами здатен розділити вхідний простір на до 2^m класів за умови лінійної роздільності даних.

У другій частині роботи було продемонстровано важливість нормування вхідних даних. Використання нормованої функції навчання 'learnpr' та попереднє масштабування ознак до діапазону $[0, 1]$ суттєво скорочують кількість ітерацій та підвищують стабільність навчання — особливо у випадках, коли ознаки мають різні масштаби (наприклад, кількість героїв і кількість сторінок у задачі класифікації літературних творів).

Завдання 3 другої частини наочно показало обмеження одношарового персептрона: просторово складний розподіл класів, де один клас оточує інший, не може бути розділений однією гіперплощиною. Проте використання трьох нейронів дозволяє апроксимувати таку межу, які разом утворюють складнішу зону класифікації.

Таким чином, лабораторна робота закріпила розуміння архітектури, процесу навчання та обмежень одношарових персептронів, а також навички роботи з функціями `perceptron`, `newpr`, `train`, `sim`, `plotpv` та `plotpc` для побудови та візуалізації нейромережових класифікаторів.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Руденко О.Г., Бодянський Є.В. Штучні нейронні мережі: Навчальний посібник. – Київ : Компанія СМІТ, 2006. – 404 с.
2. Тимощук П.В. Штучні нейронні мережі: Навчальний посібник. – Львів : Видавництво Львівської політехніки, 2011. – 444 с.
3. Уосермен Ф. Нейрокомп'ютерна техніка: Теорія і практика / Пер. з англ. І.Ю. Юрчак, 2001. [Електронний ресурс] – URL: <http://www.victoria.lviv.ua/html/wosserman/index.htm>
4. Михайленко В. М. Нейромережеві моделі та методи розпізнавання фонем в голосовому сигналі в системі дистанційного навчання :[Монографія] / В. М. Михайленко, Л. О. Терейковська, І. А.Терейковський., Б. Б. Ахметов. – К. : ЦП «Компринт», 2017.– 252 с.
5. Тимощук П. В. Штучні нейронні мережі / П. В. Тимощук - Львів. Видавництво Львівської політехніки. - 2011. – 444 с.
6. Корченко О. Методологія розроблення нейромережевих засобів інформаційної безпеки Інтернет-орієнтованих інформаційних систем: [Монографія] / О. Корченко, І. Терейковський, А. Білощицький. К. : ТОВ «Наш Формат» 2016. – 246 с.
7. Субботін С. О. Нейронні мережі : теорія та практика: навч. посіб./ С. О. Субботін. – Житомир : Вид. О. О. Євенок, 2020. – 184 с